

Security Audit

Lucid Labs (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	27
Conclusion	28
Our Methodology	29
Disclaimers	31
About Hashlock	32

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Lucid Labs team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Lucid Labs is a multi-chain liquidity infrastructure protocol focused on cross-chain asset transfers, stablecoin liquidity, and bridge aggregation for blockchain ecosystems. The platform provides a unified "Multi-Bridge" architecture that integrates multiple interoperability solutions, allowing projects to transfer assets across numerous chains through a single interface while reducing bridge fragmentation and vendor lock-in. Lucid also offers liquidity-focused products such as Nebula, a yield-generating stablecoin infrastructure, and Vested Emission Offerings (VEOs), designed to support liquidity bootstrapping and ecosystem growth. The protocol supports multiple blockchain networks and emphasizes modular cross-chain messaging, bridge abstraction, and institutional-grade security for decentralized finance.

Project Name: Lucid Labs

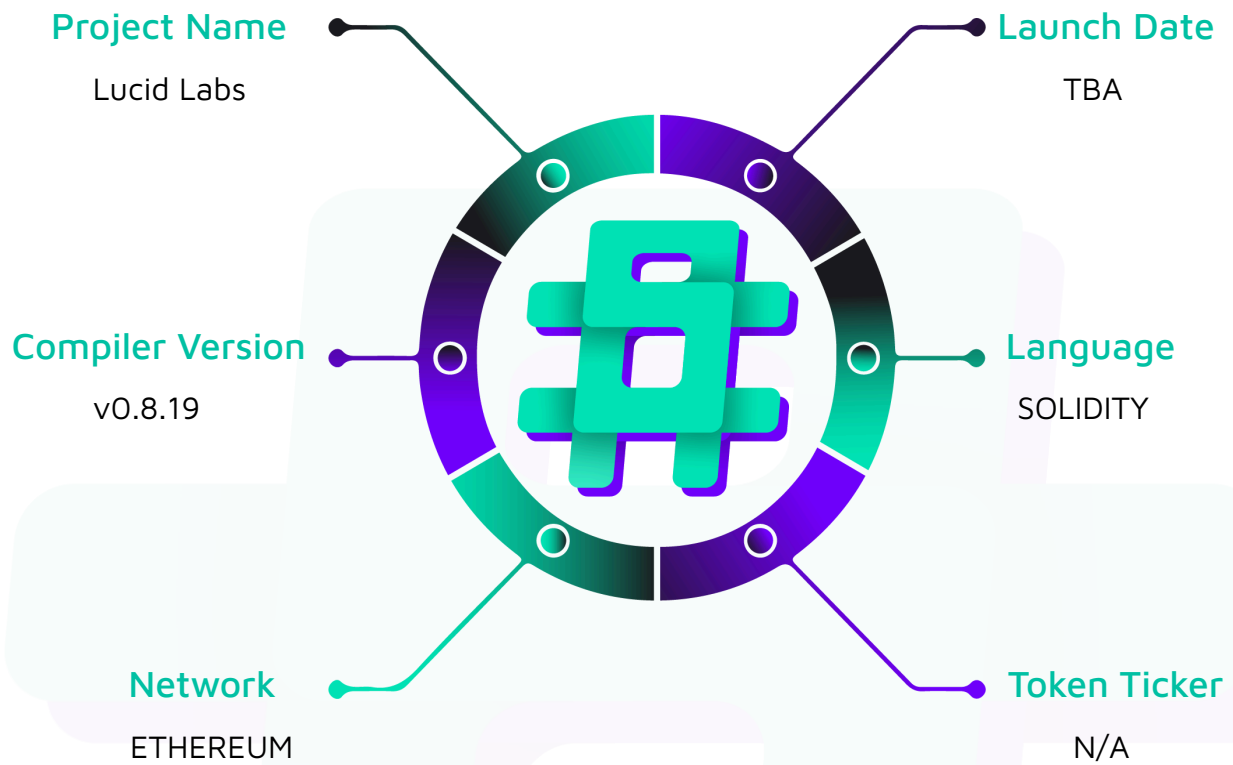
Project Type: DeFi

Compiler Version: 0.8.19

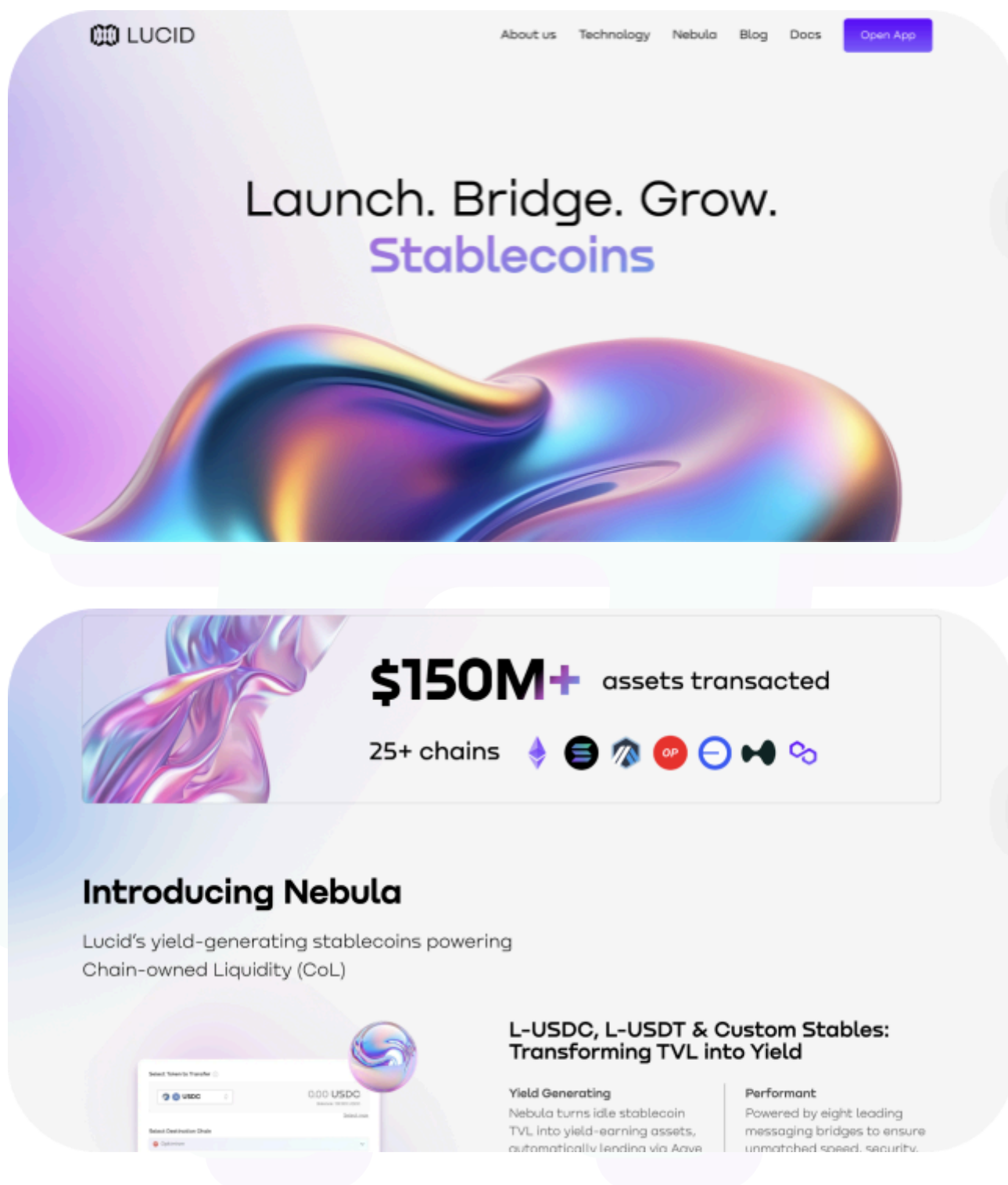
Website: <https://lucidlabs.fi/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Lucid Labs project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Lucid Labs Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	May, 2026
Contract 1	AssetController.sol
Audited GitHub Commit Hash	83ffa963889d887336595e175a4a0a31e0881be5
Fix Review GitHub Commit Hash	08678f46fe33e38b18fbf802e88d9bb2518de7f5

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Hashlocked"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerability

1 Low severity vulnerability

5 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
AssetController.sol Allows admins to: - Configure controllers, adapters, limits, pauses, and unwrapping - Withdraw accumulated native ETH refunds Allows pause roles to: - Globally pause or unpause the controller Allows users to: - Bridge assets through one or multiple adapters - Resend created transfers - Execute ready multi-bridge transfers Allows bridge adapters to: - Deliver inbound transfer messages	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Lucid Labs project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Lucid Labs project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] `minBridges == 1` makes multi-bridge transfers route through the single-bridge receive path

Description

The multi-bridge `transferTo` overload creates transfers with `threshold = minBridges`. If `minBridges` is configured to 1, the produced transfer has `threshold == 1`. On the destination chain, `receiveMessage` uses `transfer.threshold == 1` to select the single-bridge execution branch, so a transfer initiated through the multi-bridge API is processed as a single-bridge transfer.

Vulnerability Details

The multi-bridge send path stores the live `minBridges` value as the transfer threshold:

```
Transfer memory transfer = _createTransfer(destChainId, recipient, amount, unwrap, minBridges);
```

The destination receives the path, then treats every `threshold == 1` transfer as a single-bridge:

```
if (transfer.threshold == 1) {  
    if (mintingMaxLimitOf(msg.sender) == 0) revert Controller_AdapterNotSupported();  
    ...  
    _useMinterLimits(msg.sender, transfer.amount);  
    _mint(transfer.recipient, transfer.amount);  
} else {  
    if (!multiBridgeAdapters[msg.sender]) revert Controller_AdapterNotSupported();  
    ...  
}
```

Therefore, when `minBridges == 1`, the transfer does not use the multi-bridge receive

branch, does not check `multiBridgeAdapters[msg.sender]`, and does not use the global multi-bridge minting bucket at execution time.

Impact

This is a configuration-sensitive semantic break in the multi-bridge flow. If operators set `minBridges == 1`, transfers submitted through the multi-bridge API can behave differently from what the API name and accounting model imply:

- If the delivering adapter has no single-bridge minting limit, the transfer reverts on destination even though the adapter may be configured for multi-bridge delivery.
- If the delivering adapter has a single-bridge minting limit, the transfer executes immediately through per-adapter single-bridge accounting instead of accumulating multi-bridge confirmations and using the global multi-bridge bucket.

Under a trusted-admin and trusted-adapter model, this is not an arbitrary external exploit. The risk is that a valid admin configuration value silently collapses multi-bridge semantics into the single-bridge path, causing unexpected failed transfers or weaker-than-intended accounting and confirmation behavior.

Recommendation

Disallow `minBridges == 1` anywhere multi-bridge mode is configured. Treat `0` as disabled and require `minBridges >= 2` for enabled multi-bridge consensus.

Apply the invariant in both the constructor and `setMinBridges`:

```
if (_minBridges == 1) revert Controller_Invalid_Params();
```

Also, update NatSpec and operational documentation to state that multi-bridge mode is either disabled with `0` or enabled with at least two required bridge confirmations.

Status

Resolved

Low

[L-01] AssetController constructor - should validate the default admin address

Description

AssetController validates the token address in `_addresses[0]`, but does not validate `_addresses[1]`, which is used as the initial default admin. If `_addresses[1]` is accidentally set to `address(0)`, the deployed controller will not have a usable admin account.

Vulnerability Details

AssetController passes `_addresses[1]` to BaseAssetBridge, which grants `DEFAULT_ADMIN_ROLE` without checking for `address(0)`.

```
_setupRole(DEFAULT_ADMIN_ROLE, _owner);  
_setupRole(PAUSE_ROLE, _owner);  
_grantRole(PAUSE_ROLE, _pauser);
```

If `_addresses[1]` is accidentally set to `address(0)`, the deployed controller will not have a usable default admin. Admin-only functions such as `setLimits`, `setControllerForChain`, `setMultiBridgeAdapters`, and `withdraw` would not be callable by any live account.

The separate pauser address is less critical because `_owner` also receives `PAUSE_ROLE`. The selector inputs are also intentionally flexible so the controller can support different token standards and derived controllers that override token movement. Therefore, this finding is limited to the missing non-zero validation for the default admin address.

Impact

This is a deployment-time configuration risk. It does not create a direct exploit path for external users, but an accidental zero admin address can permanently lock the controller's admin functionality and require redeployment.

Recommendation

Validate `_addresses[1]` against `address(0)` in the constructor before granting roles.

```
if (_addresses[1] == address(0)) revert Controller_Invalid_Params();
```

Keep selector handling flexible, but document that the base `AssetController` must be deployed with token-compatible mint and burn selectors, while derived controllers that override `_mint` and `_burn` may intentionally ignore those selector values.

Status

Resolved

QA

[Q-01] AssetController single-bridge - replay guard should use `executed` instead of `amount`

Description

The single-bridge replay guard in `AssetController.receiveMessage`` used `receivedTransfers[transfer.transferId].amount != 0`` to determine whether a transfer had already been processed, even though the `ReceivedTransfer`` struct already stores an explicit `executed`` boolean.

Valid source-side transfers cannot have `amount == 0``, because the source transfer functions reject zero-amount transfers before creating and relaying a transfer. Therefore, this did not create a practical replay path for legitimate transfers. The issue is a defensive consistency concern: replay protection should rely on the dedicated execution state rather than an incidental transfer data field.

Vulnerability Details

In the single-bridge branch, the previous replay check was based on the stored amount:

```
if (receivedTransfers[transfer.transferId].amount != 0) revert
Controller_TransferNotExecutable();
receivedTransfers[transfer.transferId] = ReceivedTransfer({
  recipient: transfer.recipient,
  amount: transfer.amount,
  unwrap: transfer.unwrap,
  receivedSoFar: 1,
  threshold: 1,
  originChainId: originChain,
  executed: true
});
```

However, the same branch stores `executed: true`` after processing the transfer. Using `amount`` as an implicit sentinel made the receive-side state model less precise and left an unnecessary edge case for invalid zero-amount inbound payloads.

Impact

Under the protocol assumption that whitelisted adapters and admin-configured routes are trusted, this does not create a practical fund-loss issue. Legitimate source-originated transfers cannot have zero amount. the concern is limited to defensive consistency and clearer replay-state handling.

Recommendation

Use the dedicated `executed` field as the replay sentinel:

```
if (receivedTransfers[transfer.transferId].executed) revert  
Controller_TransferNotExecutable();
```

Status

Resolved

[Q-02] Cross-chain routing configuration changes have no grace period for in-flight messages

Description

`AssetController.receiveMessage` authenticates inbound messages against live routing configuration. Admin changes such as rotating `controllerForChain` or removing a multi-bridge adapter take effect immediately, with no grace period for messages that were already in flight.

Under the protocol assumption that admin roles are trusted, this is an operational hardening issue rather than an external exploit path.

Vulnerability Details

Inbound delivery checks the live controller mapping and multi-bridge adapter whitelist:

```
address controllerForChain = getControllerForChain(originChain);
if (controllerForChain == address(0) || controllerForChain != originSender) revert
Controller_Invalid_Params();
if (!multiBridgeAdapters[msg.sender]) revert Controller_AdapterNotSupported();
```

The corresponding admin setters apply changes immediately:

```
function setControllerForChain(uint256[] memory chainId, address[] memory controller)
public onlyRole(DEFAULT_ADMIN_ROLE) {
    _setControllerForChain(chainId, controller);
}
```

Because there is no pending state, activation delay, or bounded overlap period, messages that were valid when sent can fail after configuration changes. For example, messages from a previous controller can fail after controller rotation, and partially delivered multi-bridge transfers can fail if a required adapter is removed before the threshold is reached.

Impact

Admin-triggered configuration changes can temporarily block legitimate in-flight transfers until operators restore the prior configuration, re-add an adapter, or route a



resend through currently valid adapters.

No arbitrary external caller can trigger this condition, and under the trusted-admin model this does not create a direct fund-loss vulnerability. The main risk is operational friction during controller migrations.

Recommendation

Document a routing-change runbook that pauses or avoids new outbound transfers, waits for in-flight messages to settle, and only then updates controller or adapter configuration.

For stronger migration safety, consider two-step configuration changes with an activation delay or a bounded grace period for already in-flight messages.

Note

Lucid Labs informed Hashlock that cross-chain configuration updates are managed exclusively by internal operators and covered by internal SOPs for message-delivery-impacting changes. No public NatSpec or documentation update is planned.

Status

Acknowledged

[Q-03] Asset transfers - do not define an expiry or user-side recovery path

Description

`AssetController` stores relayed asset transfers and allows messages to be re-sent, but it does not define an expiry, cancellation, refund, or timeout flow for asset transfers that remain blocked on the destination chain.

Under the trusted-admin model, this is a recovery-process limitation rather than a direct vulnerability.

Vulnerability Details

Source-side transfers burn tokens before relaying the destination message:

```
_burn(_msgSender(), amount);
```

The transfer is then stored in `relayedTransfers` and can be retried through `resendTransfer`, but the `Transfer` struct has no deadline or cancellation state, and `AssetController` does not expose a `cancelTransfer`, `refundTransfer`, or `timeoutTransfer` function.

This means that if destination execution remains blocked by configuration, adapter, or operational issues, users cannot self-serve a source-side recovery from the asset controller itself.

Impact

Transient delivery failures can often be handled with `resendTransfer`. However, if a transfer remains blocked until admin intervention, users depend on operational coordination or a manual compensating action outside the normal transfer flow.

Recommendation

Document the recovery model clearly: asset transfers are not cancellable by users after the source-side burn, and recovery from permanently blocked destination execution requires operator/admin coordination.

Include an operational runbook for stuck transfers covering when to retry via `resendTransfer`, when to restore configuration temporarily, and how admins should

handle exceptional compensating mints or manual recovery. If protocol requirements demand user-side recovery, consider a carefully designed timeout or cancellation mechanism with safeguards against double minting.

Note

Lucid Labs added NatSpec, clarifying that transfers are not cancellable or refundable after source-side burn/lock, and stated that stuck-transfer handling is managed internally while `resendTransfer` remains the documented retry mechanism

Status

Acknowledged

[Q-04] AssetController#receiveMessage - Documented dual-use adapter trust model

Description

AssetController supports both single-bridge and multi-bridge message delivery. In the single-bridge flow, a whitelisted adapter with a configured minting limit is trusted to deliver messages and mint up to its per-day limit. In the multi-bridge flow, higher limits can additionally require multiple bridge adapters to confirm the same message.

The same adapter can be configured for both modes. In that case, the adapter retains its unilateral single-bridge allowance while also being able to participate in higher-threshold multi-bridge transfers. This is an intentional additive trust model rather than a restriction that prevents dual-use adapters from using the single-bridge path.

Vulnerability Details

The single-bridge branch is selected by `transfer.threshold == 1` and authorizes the caller by checking whether it has a non-zero minting limit.

```
if (transfer.threshold == 1) {  
    if (mintingMaxLimitOf(msg.sender) == 0) revert Controller_AdapterNotSupported();  
    ...  
    _useMinterLimits(msg.sender, transfer.amount);  
    _mint(transfer.recipient, transfer.amount);  
}
```

The multi-bridge branch is reached when `transfer.threshold > 1`, where `msg.sender` must be present in `multiBridgeAdapters`. Because dual-use adapters are intentionally supported, an adapter configured in both roles can process lower-limit transfers through the single-bridge path and participate in higher-limit transfers through the multi-bridge path.

Impact

This does not let an arbitrary external account mint tokens. The caller must be a trusted/whitelisted adapter path or an address granted adapter limits. Under the

documented trust model, a single-bridge adapter's per-day minting limit is the intended risk boundary for unilateral delivery, while higher multi-bridge limits remain protected by the configured bridge confirmation threshold.

Recommendation

No code change is required if this trust model is intended. Continue documenting that whitelisted single-bridge adapters are trusted to deliver messages and mint up to their configured per-day limits, and that higher limits are available through multi-bridge confirmation. Operationally, keep single-bridge limits conservative relative to the risk of each adapter and reserve higher limits for multi-bridge flows.

Status

Resolved

[Q-05] Missing event emission on `withdraw`

Description

`AssetController.withdraw` transfers the contract's full ETH balance to an admin-chosen recipient but does not emit an event. The contract can accumulate ETH through its payable `receive` function, which is documented as receiving bridge refund ETH.

Vulnerability Details

The withdrawal function performs the transfer without recording a protocol-level event:

```
function withdraw(address payable recipient) public onlyRole(DEFAULT_ADMIN_ROLE) {  
    if (recipient == address(0)) revert Controller_Invalid_Params();  
    (bool success, ) = recipient.call{value: address(this).balance}("");  
    if (!success) revert Controller_EtherTransferFailed();  
}
```

The fallback accepts ETH:

```
receive() external payable {}
```

Without an event, off-chain monitoring systems must infer withdrawals from native ETH balance changes rather than a dedicated controller event.

Impact

Admin ETH sweeps are less visible to protocol monitoring, treasury dashboards, and incident-response tooling. This can make it harder to distinguish routine refund collection from unexpected movement after an admin-key compromise.

Recommendation

Emit a withdrawal event that records the recipient and the amount withdrawn:

```
event Withdrawal(address indexed recipient, uint256 amount);  
function withdraw(address payable recipient) public onlyRole(DEFAULT_ADMIN_ROLE) {  
    if (recipient == address(0)) revert Controller_Invalid_Params();  
    uint256 amount = address(this).balance;  
    (bool success, ) = recipient.call{value: amount}("");  
}
```

```
if (!success) revert Controller_EtherTransferFailed();  
emit Withdrawal(recipient, amount);  
}
```

Status

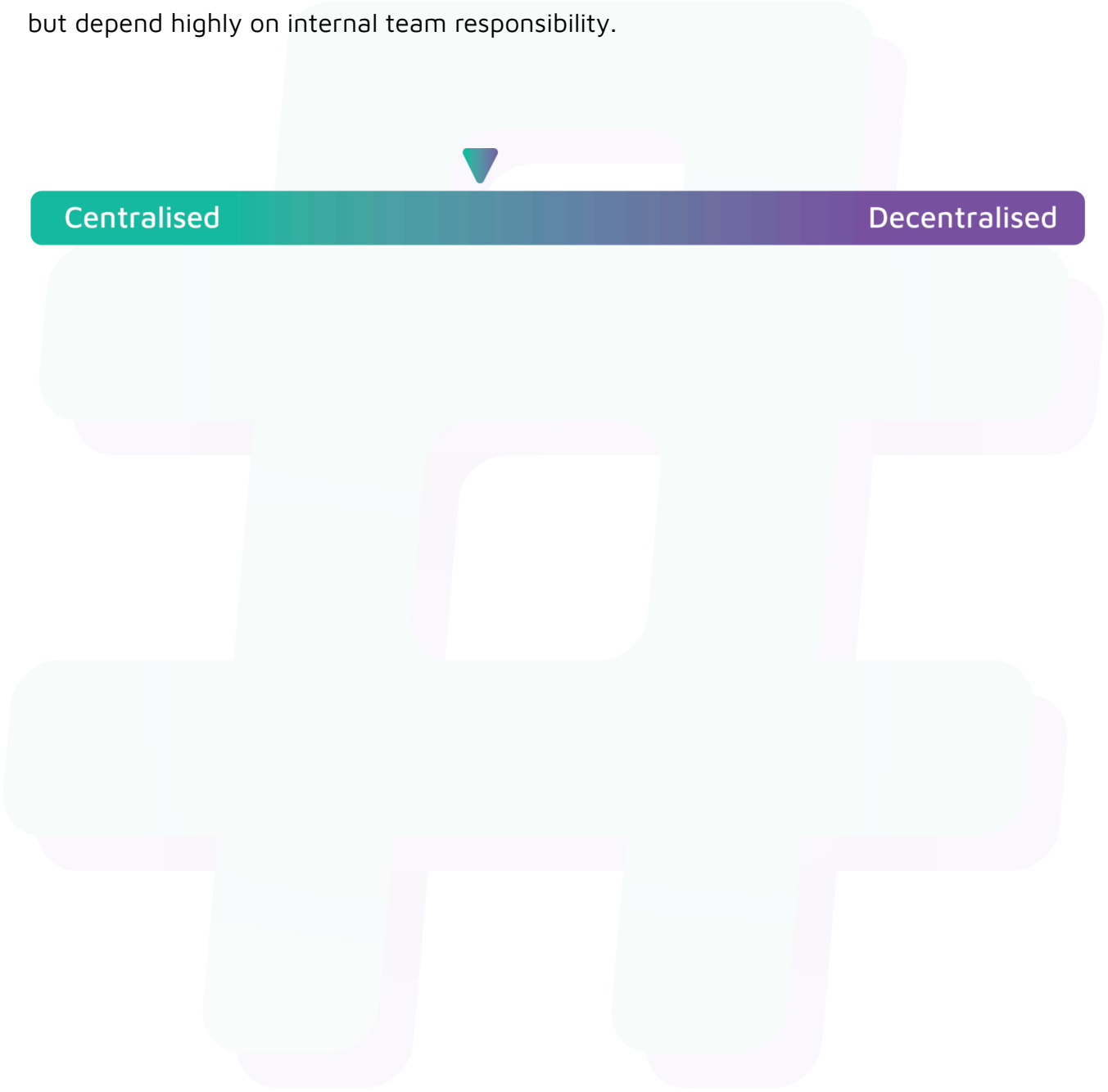
Resolved



Centralisation

The Lucid Labs project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Lucid Labs project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.